

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

«Программирование»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Объектно-ориентированное программирование в Python

Выполнил:

Студент группы N3150

Пахомов Владимир Юрьевич



Проверил:

Грозов В. А.

Санкт-Петербург

2025г.

Оглавление

Задание. Вариант 4-1.....	3
Пример работы программы при различных исходных данных.....	4
Исходный текст программы с комментариями.....	5

Задание. Вариант 4-1

Таблица 1:

4 — dict (ограничение формата накладывается на ключи словаря)

Таблица 2:

1 — IPv4-адрес Десятичный формат с точками

Пример работы программы при различных исходных данных

```
>>> from prg5vypN3150 import MyDict
>>> d = MyDict({'1.1.1.1': 10, '2.2.2.2': 20, '3.3.3.3': 30})
>>> d.update({'4.4.4.4': 40})
>>> del d['1.1.1.1']
>>> d
{'2.2.2.2': 20, '3.3.3.3': 30, '4.4.4.4': 40}
>>> d |= [(192.168, 21), ('192.168.0.1', 321)]
Traceback (most recent call last):
  File "<python-input-5>", line 1, in <module>
    d |= [(192.168, 21), ('192.168.0.1', 321)]
  File "/home/vladimirp/C_Workspace/prg5vypN3150/prg5vypN3150.py", line 90, in __ior__
    self.check_type(iterable)
    ~~~~~^~~~~~
  File "/home/vladimirp/C_Workspace/prg5vypN3150/prg5vypN3150.py", line 53, in check_type
    raise TypeError('Ошибка типа данных')
TypeError: Ошибка типа данных
>>> d |= [('192.168.10.10', 21), ('192.168.0.1', 321)]
>>> d
{'2.2.2.2': 20, '3.3.3.3': 30, '4.4.4.4': 40, '192.168.10.10': 21, '192.168.0.1': 321}
>>> d | {'5.5.5.5': 50}
{'2.2.2.2': 20, '3.3.3.3': 30, '4.4.4.4': 40, '192.168.10.10': 21, '192.168.0.1': 321, '5.5.5.5': 50}
>>> d
{'2.2.2.2': 20, '3.3.3.3': 30, '4.4.4.4': 40, '192.168.10.10': 21, '192.168.0.1': 321}
>>> d['2.2.2.2'] = 202
>>> d
{'2.2.2.2': 202, '3.3.3.3': 30, '4.4.4.4': 40, '192.168.10.10': 21, '192.168.0.1': 321}
>>> d.undo()
>>> d.undo()
>>> d
{'2.2.2.2': 20, '3.3.3.3': 30, '4.4.4.4': 40}
>>> d.redo()
>>> d
{'2.2.2.2': 20, '3.3.3.3': 30, '4.4.4.4': 40, '192.168.10.10': 21, '192.168.0.1': 321}
>>> d.redo()
>>> d.redo()
Traceback (most recent call last):
  File "<python-input-18>", line 1, in <module>
    d.redo()
    ~~~~~^
  File "/home/vladimirp/C_Workspace/prg5vypN3150/prg5vypN3150.py", line 71, in redo
    raise RedoError()
prg5vypN3150.RedoError: Невозможно выполнить redo
>>> d.undo()
>>> d.undo()
>>> d.undo()
>>> d.undo()
>>> d.undo()
Traceback (most recent call last):
  File "<python-input-23>", line 1, in <module>
    d.undo()
    ~~~~~^
  File "/home/vladimirp/C_Workspace/prg5vypN3150/prg5vypN3150.py", line 62, in undo
    raise UndoError()
prg5vypN3150.UndoError: Невозможно выполнить undo
>>> d
{'1.1.1.1': 10, '2.2.2.2': 20, '3.3.3.3': 30}
>>> d.clear()
>>> d
{}
>>> d.undo()
>>> d
{'1.1.1.1': 10, '2.2.2.2': 20, '3.3.3.3': 30}
>>> █
```

Исходный текст программы с комментариями

Файл prg5vypN3150.py:

```
class FormatError(Exception):
    def __init__(self, message='Неверный формат IPv4'):
        super().__init__(message)

class UndoError(Exception):
    def __init__(self, message='Невозможно выполнить undo'):
        super().__init__(message)

class RedoError(Exception):
    def __init__(self, message='Невозможно выполнить redo'):
        super().__init__(message)

# Необходимо выбрать один из классов (см. Табл. 1 в задании)

#class MyList(list):
#class MySet(set):
#class MyDict(dict):

class MyDict(dict):
    # Тут ваш код

    def __init__(self, iterable={}):
        super().__init__(iterable)
        self.check_type(iterable)
        self.undo_stack = [super().copy()]
        self.redo_stack = []

    '''Проверка формата строки'''
    def check_format(self, value):
        import re
        part = r'((25[0-5])|(2[0-4]\d)|(1\d\d)|([1-9]?d))'
        if not isinstance(value, str):
            raise TypeError()
        if not re.match(fr'^{part}\.{part}\.{part}\.{part}$', value):
            raise FormatError()

    '''Проверка типа данных'''
    def check_type(self, iterable):
        if isinstance(iterable, dict):
            for key in iterable.keys():
                if isinstance(key, str):
                    self.check_format(key)
                else:
                    raise TypeError('Ошибка типа данных')
        elif isinstance(iterable, list) or isinstance(iterable, tuple) or isinstance(iterable, set):
            for el in iterable:
                if isinstance(el, tuple) and len(el) == 2:
                    if isinstance(el[0], str):
                        self.check_format(el[0])
                    else:
                        raise TypeError('Ошибка типа данных')
                else:
                    raise TypeError('Ошибка типа данных')
            else:
                raise TypeError('Ошибка типа данных')

    '''Команда undo'''
    def undo(self):
        if len(self.undo_stack) == 1:
```

```

        raise UndoError()
        self.redo_stack.append(self.undo_stack[-1].copy())
        del self.undo_stack[-1]
        super().clear()
        super().update(self.undo_stack[-1].copy())

'''Команда redo'''
def redo(self):
    if len(self.redo_stack) == 0:
        raise RedoError()
    self.undo_stack.append(self.redo_stack[-1].copy())
    del self.redo_stack[-1]
    super().clear()
    super().update(self.undo_stack[-1].copy())

'''Переопределение методов словаря'''
def __setitem__(self, key, value):
    self.check_format(key)
    super().__setitem__(key, value)
    self.undo_stack.append(super().copy())
    self.redo_stack.clear()

def __delitem__(self, key):
    super().__delitem__(key)
    self.undo_stack.append(super().copy())
    self.redo_stack.clear()

def __ior__(self, iterable):
    self.check_type(iterable)
    super().update(iterable)
    self.undo_stack.append(super().copy())
    self.redo_stack.clear()
    return self

def __or__(self, iterable):
    self.check_type(iterable)
    new = MyDict()
    new.update(self.copy())
    new.undo_stack = self.undo_stack.copy()
    new.redo_stack = self.redo_stack.copy()
    new.update(iterable)
    return new

def update(self, iterable):
    self.check_type(iterable)
    super().update(iterable)
    self.undo_stack.append(super().copy())
    self.redo_stack.clear()

def pop(self, key):
    del_key = super().pop(key)
    self.undo_stack.append(super().copy())
    self.redo_stack.clear()
    return del_key

def popitem(self):
    del_item = super().popitem()
    self.undo_stack.append(super().copy())
    self.redo_stack.clear()
    return del_item

def setdefault(self, key, default):
    self.undo_stack.append(super().copy())
    super().setdefault(key, default)
    self.redo_stack.clear()
    return default

def clear(self):

```

```
super().clear()
self.undo_stack.append(super().copy())
self.redo_stack.clear()
```

```
# Вариант 4-1
# 4 - dict (ограничение на ключи словаря)
# 1 - IPv4
```

Файл test_prg5vypN3150.py

```
from prg5vypN3150 import FormatError, UndoError, RedoError, MyDict      # Импортировать ваш
класс
import pytest

# Тут ваши тесты

def test_init_dict():
    d1 = MyDict({'192.168.0.1': 12, '8.8.8.8': True})
    d2 = MyDict([('255.255.255.0', 'Hello'), ('0.0.0.0', 123.21)])
    d3 = MyDict()
    assert d1['192.168.0.1'] == 12
    assert d2['0.0.0.0'] == 123.21
    assert d3 == {}

def test_check_format():
    with pytest.raises(FormatError):
        d = MyDict({'123.355.3.1': 213})
        d = MyDict({'123.355.3.1.1'})

def test_check_type():
    with pytest.raises(TypeError):
        d = MyDict([(255.12, 90)])
        d = MyDict(['2.2.2.2', False])
        d = MyDict(12390)

def test_setitem():
    d = MyDict({'192.168.0.1': 12, '8.8.8.8': True})
    d['8.8.8.8'] = 5
    d['7.7.7.7'] = 90
    assert d['8.8.8.8'] == 5
    assert d['7.7.7.7'] == 90
    with pytest.raises(FormatError):
        d['256.0.0.0'] = 21

def test_ior_and_or():
    d = MyDict({'192.168.0.1': 12, '8.8.8.8': True})
    d |= {'5.5.5.5': 321}
    assert '5.5.5.5' in d
    d |= {'4.4.4.4': 432}
    assert '4.4.4.4' not in d

def test_pop_and_popitem():
    d = MyDict({'192.168.0.1': 12, '8.8.8.8': True, '5.5.5.5': False})
    del_key = d.pop('8.8.8.8')
    assert del_key not in d
    assert del_key == True
    del_item = d.popitem()
    assert ('5.5.5.5', False) == del_item

def test_update_and_setdefault():
    d = MyDict({'192.168.0.1': 12, '8.8.8.8': True, '5.5.5.5': False})
    d.update([('2.2.2.2', '1'), ('1.2.3.4', '3')])
    assert '2.2.2.2' in d and '1.2.3.4' in d
    d.setdefault('8.8.8.8', 'True')
    d.setdefault('9.8.7.6', 'True')
    assert d['8.8.8.8'] == True
    assert d['9.8.7.6'] == 'True'

def test_delitem_and_clear():
    d = MyDict({'192.168.0.1': 12, '8.8.8.8': True, '5.5.5.5': False})
    del d['8.8.8.8']
    assert '8.8.8.8' not in d
```



```
d.clear()
assert len(d) == 0

def test_undo_and_redo():
    d = MyDict({'192.168.0.1': 12, '8.8.8.8': True, '5.5.5.5': False})
    cp = d.copy()
    del d['8.8.8.8']
    cp_r = d.copy()
    d.undo()
    assert cp == d
    d.redo()
    assert cp_r == d
    with pytest.raises(UndoError):
        d.undo()
        d.undo()
    with pytest.raises(RedoError):
        d.redo()
        d.redo()
```

```
# Количество тестов выбирайте самостоятельно,
# но так, чтобы вся необходимая функциональность
# была проверена.
```